

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені Тараса Шевченка
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
Кафедра програмних систем і технологій

Дисципліна
«Проектування баз даних»
Лектор Духновська Ксенія Костянтинівна
Лабораторна робота № 7

Виконав студент ІПЗ – 23
Тимчук Артур Володимирович

14 травня 2026

Підготовка бази даних

Перед початком тестування я вирішив підготувати базу та додати елементи, які гарантуватимуть цілісність даних та виконання бізнес-логіки, а саме: обмеження (constraints), тригери та збережені процедури. Оскільки база даних реалізована на PostgreSQL, я використовував діалект PL/pgSQL.

Почав я з модифікації схеми:

- додано явні DEFAULT-значення: `Is_Active = TRUE` для таблиць `Menu` та `Recipe`, `Current_Status = 'New'` для `Customer_Order`.
- додано CHECK-обмеження: `chk_min_norm_positive` (заборона від'ємної мінімальної норми на складі) та `chk_prices_positive` (заборона від'ємної ціни для страв).
- додано NOT NULL до необхідних колонок.

З тригерів я додав наступні для відповідності бізнес-правилам:

- `trg_prevent_future_supply`: Заборона вказання майбутньої дати при оформленні постачання.
- `trg_log_order_status`: Автоматичне логування у таблицю `Order_Status_Log` при будь-якій зміні статусу замовлення.
- `trg_prevent_active_menu_deletion`: Заборона видалення запису з таблиці `Menu`, якщо воно наразі є активним.

Процедури я додав наступні (усі сценарії реалізовані з обробкою помилок `RAISE EXCEPTION` для забезпечення ACID):

1. `usp_RegisterEmployee` - реєстрація нового працівника з валідацією логіну та довжини пароля, а також перевіркою на дублікати.
2. `usp_CreateCustomerOrder` – створення замовлення з атомарною перевіркою доступності столика та автоматичною зміною його статусу на 'Occupied'.
3. `usp_ChangeOrderStatus` - зміна статусу замовлення з перевіркою, чи не збігається новий статус з поточним (щоб уникнути дублювання логів).

Планування структурного тестування

Для тестування я обрав фреймворк **pgTAP** — розширення для модульного тестування у PostgreSQL. Усі тести загорнуті у блок **BEGIN; ... ROLLBACK;**, що гарантує повну ізоляцію та незмінність робочої бази даних після тестів (ACID - Ізоляція).

Тести розділено на класи:

- **ST_Schema / ST_Keys** - перевірка схеми: таблиці, ключі, NOT NULL.
- **ST_Constraints** - CHECK та UNIQUE обмеження.
- **ST_Triggers** - логіка спрацювання тригерів.
- **ST_Procedures** - логіка процедур, перевірка валідації (IF-блоків).

Структурне тестування

Спочатку була перевірка схеми, ключів, обмежень та внутрішньої валідації процедур. Після чого відбулась перевірка тригерів та процедур.

```
-- -----  
-- ST_Schema & ST_Keys: Structure and Schema Tests  
-- -----  
  
SELECT has_table('employee', 'ST_Schema: Table Employee exists');  
  
SELECT has_table('customer_order', 'ST_Schema: Table Customer_Order  
exists');  
  
SELECT col_not_null('employee', 'full_name', 'ST_Schema: Employee Full_Name  
is NOT NULL');  
  
SELECT col_has_default('menu', 'is_active', 'ST_Schema: Menu Is_Active has a  
default value');  
  
SELECT has_pk('restaurant_table', 'ST_Keys: Restaurant_Table has a Primary  
Key');  
  
SELECT has_fk('employee', 'ST_Keys: Employee table has Foreign Keys');  
  
-- -----  
-- ST_Constraints: Check constraints
```

```

-----
SELECT throws_ok(
    'INSERT INTO Product (Product_Name, Min_Norm) VALUES (''Test
Product'', -5)',
    '23514', -- PostgreSQL error code for check_violation
    NULL,
    'ST_Constraints: chk_min_norm_positive prevents negative
minimum norms'
);

```

```

SELECT throws_ok(
    'INSERT INTO Dish (Category_ID, Dish_Name, Cost_Price,
Markup_Percent, Current_Price) VALUES (1, ''Test Dish'', -10, 20, 15)',
    '23514',
    NULL,
    'ST_Constraints: chk_prices_positive prevents negative dish
prices'
);

```

```

-----
-- ST_Triggers: Trigger Logic Tests
-----

```

```

SELECT has_trigger('supply', 'trg_prevent_future_supply', 'ST_Triggers:
Supply has future date prevention trigger');

```

```

SELECT throws_ok(
    $$INSERT INTO Supply (Warehouse_ID, Supplier_ID,
Invoice_Number, Supply_Date) VALUES (1, 1, 'INV-001', CURRENT_DATE + INTERVAL
'1 day')$$,
    'P0001',
    'Supply_Date cannot be in the future.',

```

```

        'ST_Triggers: Future supply dates are successfully blocked'

    );

-- =====
-- PREPARATION OF DATA FOR LOGIC TESTING
-- =====

-- Create base roles and employee (Using extremely high IDs to avoid
collision with real data)

INSERT INTO Role (Role_ID, Role_Name, Access_Level) VALUES (9999999, 'Test
Role', 1);

INSERT INTO Position (Position_ID, Position_Name, Base_Salary) VALUES
(9999999, 'Test Pos', 1000);

INSERT INTO Employee (Employee_ID, Role_ID, Position_ID, Full_Name, Login,
Password_Hash)
VALUES (9999999, 9999999, 9999999, 'Mock Action User', 'mockaction',
'hash123');

-- Mock data for testing order CREATION (Table 9999998)

INSERT INTO Restaurant_Table (Table_ID, Table_Number, Status) VALUES
(9999998, 9999998, 'Available');

-- Mock data for testing order STATUS CHANGE (Table 9999999 and Order
9999999)

INSERT INTO Restaurant_Table (Table_ID, Table_Number, Status) VALUES
(9999999, 9999999, 'Occupied');

INSERT INTO Customer_Order (Order_ID, Table_ID, Employee_ID, Current_Status)
VALUES (9999999, 9999999, 9999999, 'New');

-- -----
-- ST_Procedures: Structural logic checks (IF blocks, validation)
-- -----

SELECT throws_ok(

```

```

        $$CALL usp_RegisterEmployee(9999999, 9999999, 'Test User', '',
'hashed_pass')$$,

        'P0001',

        'Login cannot be empty.',

        'ST_Procedures: usp_RegisterEmployee catches empty login'

    );

SELECT throws_ok(

        $$CALL usp_RegisterEmployee(9999999, 9999999, 'Test User',
'testlogin', '123')$$,

        'P0001',

        'Password hash must be at least 6 characters.',

        'ST_Procedures: usp_RegisterEmployee catches short passwords'

    );

-- Now we know the exact Order_ID (9999999), so we call it directly!

SELECT throws_ok(

        $$CALL usp_ChangeOrderStatus(9999999, 9999999, 'New')$$,

        'P0001',

        'New status is the same as the current status.',

        'ST_Procedures: usp_ChangeOrderStatus prevents identical status
update'

    );

```

Результати структурного тестування:

Назва тесту	Клас	Що перевіряє	Очікуваний результат	Результат

has_table_employee	ST_Schema	Наявність таблиці Employee	Всі таблиці присутні	ok
has_table_customer_order	ST_Schema	Наявність таблиці Customer_Order	Всі таблиці присутні	ok
col_not_null_employee	ST_Schema	NOT NULL для поля full_name	Поле не може бути NULL	ok
col_has_default_menu	ST_Schema	DEFAULT значення для is_active	Default присутній	ok
has_pk_restaurant_table	ST_Keys	Наявність РК у таблиці	РК присутній	ok
has_fk_employee	ST_Keys	Наявність FK у таблиці Employee	FK присутні	ok
chk_min_norm_positive	ST_Constraints	Вставка від'ємної	Помилка 23514	ok

		норми складу		
chk_prices_positive	ST_Constraints	Вставка від'ємної ціни страви	Помилка 23514	ok
trg_prevent_future_supply	ST_Triggers	Наявність тригера	Тригер існує	ok
blocks future supply dates	ST_Triggers	Вставка завтрашньої дати	Помилка 'cannot be in the future'	ok
catches empty login	ST_Procedures	IF: реєстрація з порожнім логіном	Помилка 'Login cannot be empty'	ok
catches short passwords	ST_Procedures	IF: пароль < 6 символів	Помилка 'at least 6 characters'	ok
prevents identical status update	ST_Procedures	IF: зміна на той самий статус	Помилка 'same as current status'	ok

Функціональне тестування: Чорний ящик

Тестування методом чорного ящика перевіряє відповідність поведінки системи бізнес-вимогам з точки зору кінцевого користувача (функція `lives_ok`), не враховуючи того, як саме змінюються дані "під капотом".

```
-- -----  
-- FT_BlackBox: Functional testing (user action simulation)  
-- -----  
  
SELECT lives_ok(  
    $$CALL usp_RegisterEmployee(9999999, 9999999, 'Test User',  
    'testlogin', 'securehash123')$$,  
    'FT_BlackBox: usp_RegisterEmployee succeeds with valid data'  
);  
  
SELECT throws_ok(  
    $$CALL usp_RegisterEmployee(9999999, 9999999, 'Another User',  
    'testlogin', 'securehash456')$$,  
    'P0001',  
    'Login testlogin is already taken.',  
    'FT_BlackBox: usp_RegisterEmployee blocks duplicate logins'  
);  
  
-- Use table 9999998 for the creation test  
  
SELECT lives_ok(  
    $$CALL usp_CreateCustomerOrder(9999998, 9999999, NULL)$$,  
    'FT_BlackBox: usp_CreateCustomerOrder executes without errors'  
);  
  
-- Use existing order 9999999 for the status change test  
  
SELECT lives_ok(  
    $$CALL usp_ChangeOrderStatus(9999999, 9999999, 'Processing')$$,
```

```
'FT_BlackBox: usp_ChangeOrderStatus updates successfully'

);
```

Назва тесту	Клас	Що перевіряє	Очікуваний результат	Результат
register succeeds with valid data	FT_BlackBox	Успішна реєстрація валідного працівника	Виконання без помилок	ok
register blocks duplicate logins	FT_BlackBox	Бізнес-правило: реєстрація з існуючим логіном	Помилка 'already taken'	ok
create_order executes without errors	FT_BlackBox	Створення замовлення адміністратором	Виконання без помилок	ok
change_status updates successfully	FT_BlackBox	Зміна статусу замовлення на валідний	Виконання без помилок	ok

Функціональне тестування: Білий ящик

Метод білого ящика перевіряє внутрішню логіку роботи бази даних — тобто, чи коректно змінюються стани у пов'язаних таблицях (side-effects) після виконання команд. Ми використовуємо `results_eq` та `is` для перевірки внутрішнього стану БД.

```
-----
```

```

-- FT_WhiteBox: Internal state check (DB side-effects)

-----

SELECT results_eq(

    $$SELECT Status FROM Restaurant_Table WHERE Table_ID =
9999998$$,

    $$VALUES ('Occupied'::varchar)$$,

    'FT_WhiteBox: usp_CreateCustomerOrder updates table status to
Occupied'

    );

-- Check if a log appeared for order 9999999

SELECT is(

    (SELECT COUNT(*)::int FROM Order_Status_Log WHERE Order_ID =
9999999 AND Status = 'Processing'),

    1,

    'FT_WhiteBox: trg_log_order_status automatically added a row to
the log table'

    );

```

Назва тесту	Клас	Що перевіряє	Очікуваний результат	Результат
updates table status to Occupied	FT_WhiteBox	Перевірка автоматичної зміни статусу столика процедурою	Status = 'Occupied'	ok

automatically added a row to the log table	FT_WhiteBox	Відпрацювання тригера логування після зміни статусу	1 новий рядок додано у лог	ok
--	-------------	---	----------------------------	----

Тестування під навантаженням

На цьому етапі я провів тестування під навантаженням за допомогою утиліти **pgbench** (стандартного інструменту PostgreSQL для стрес-тестування). Я задав 50 паралельних клієнтів (користувачів), що виконували транзакції протягом 60 секунд.

Для тестування я обрав запит з попередньої роботи.

```
SELECT d.Dish_Name, SUM(oi.Quantity) as Times_Ordered
FROM Dish d
      JOIN Order_Item oi ON d.Dish_ID = oi.Dish_ID
GROUP BY d.Dish_Name
ORDER BY Times_Ordered DESC
LIMIT 10;
```

Сценарій запуску у консолі:

```
pgbench -U avens -c 50 -j 10 -T 60 -f Lab7StressTest.sql restaurant
```

Результати:

Запит	Клієнтів	Потоків	Час (сек)	Всього оброблено запитів	Пропускна здатність
Lab7StressTest.sql	50	10	60	14 038	230.12 tps

Результат у понад 230 транзакцій на секунду при 50 одночасних підключеннях є високим показником для складної вибірки з агрегацією (**GROUP BY**, **ORDER BY**, **SUM**). Середня затримка склала близько 217 мс, що свідчить про стабільну роботу індексів та ефективність обробки запитів сервером бази даних навіть під час інтенсивного навантаження.

Загальний підсумок тестування

Тип тестування	Клас / Метод	Тестів	Пройдено	Результат
Структурне	ST_Schema / ST_Keys	6	6	100% ok
Структурне	ST_Constraints	2	2	100% ok
Структурне	ST_Triggers	2	2	100% ok
Структурне	ST_Procedures	3	3	100% ok
Функціональне	FT_BlackBox	4	4	100% ok
Функціональне	FT_WhiteBox	2	2	100% ok
Разом	—	19	19	100% ok

Висновок

У ході виконання лабораторної роботи проведено комплексне тестування бази даних системи управління рестораном на СУБД PostgreSQL.

Підготовка включала доопрацювання схеми (DEFAULT-значення, CHECK-обмеження), реалізацію 3 тригерів і 3 збережених процедур з ACID-гарантіями, написаними на мові PL/pgSQL.

Структурне тестування (за допомогою фреймворку **pgTAP**) підтвердило коректність схеми, ключів, обмежень та внутрішньої валідації логіки. Функціональне тестування перевірило відповідність системи бізнес-вимогам (чорний ящик) та коректність автоматичної зміни станів у БД (білий ящик).

Загальний результат: 19 тестів з 19 пройдено успішно (100%).

Навантажувальне тестування за допомогою **pgbench** (230 tps при 14 038 виконаних транзакціях) підтвердило, що база даних працює швидко і стабільно, а її архітектура готова до експлуатації під високим навантаженням.